Day 12 — SQL intermediate

MD Mutasim Billah · 52-week Data Science to ML/AI roadmap · study lesson, code to be written and verified later

1. Where junior SQL becomes senior SQL

Day 11 covered filtering, aggregating, and joining — that's most of what beginners write. Today covers the patterns that show up in every real analytics job and every SQL interview: nesting queries inside queries, conditional logic, and ranking rows within groups. Same `titanic.db` from Day 11 — no new setup needed.

2. Subqueries

A query inside a query — answer a question using the answer to another question.

```
SELECT Name, Fare
FROM titanic
WHERE Fare > (SELECT AVG(Fare) FROM titanic);
```

- The inner query runs first, returns one value, the outer query uses it
- Also works with lists: `WHERE Pclass IN (SELECT Pclass FROM class_lookup WHERE class_label != 'Third')`
- Practice: passengers who paid more than their own Pclass's average fare

3. CASE WHEN

If/else logic, inside SQL — bucket values without leaving the database.

```
SELECT Name, Age,
  CASE
    WHEN Age < 18 THEN 'Child'
    WHEN Age < 60 THEN 'Adult'
    ELSE 'Senior'
  END AS age_group
FROM titanic;
```

- Checks conditions top to bottom, stops at the first match
- `ELSE` catches everything else; omit it and unmatched rows get NULL
- Practice: label Fare as 'Budget' / 'Standard' / 'Premium'

4. CTEs — the WITH clause

Same power as a subquery, far more readable.

```
WITH class_avg AS (
  SELECT Pclass, AVG(Fare) AS avg_fare
  FROM titanic GROUP BY Pclass
)
SELECT t.Name, t.Fare, c.avg_fare
FROM titanic t
JOIN class_avg c ON t.Pclass = c.Pclass
WHERE t.Fare > c.avg_fare;
```

- `WITH name AS (...)` defines a temporary named result you can query like a table
- Same result as a nested subquery, but readable top to bottom
- Chain multiple CTEs with commas for multi-step logic

5. Window functions

ROW_NUMBER, RANK, and OVER — rank rows within groups without collapsing them.

```
SELECT Name, Pclass, Fare,
  RANK() OVER (
    PARTITION BY Pclass ORDER BY Fare DESC
  ) AS fare_rank
FROM titanic;
```

- Unlike GROUP BY, window functions keep every row — they just attach a rank/number to each
- `PARTITION BY` restarts the ranking for each group
- Practice: find the top 3 highest fares within each Pclass

6. Multiple joins

Real schemas are rarely just two tables.

```
SELECT t.Name, c.class_label, e.port_name
FROM titanic t
JOIN class_lookup c ON t.Pclass = c.Pclass
JOIN embarked_lookup e ON t.Embarked = e.Embarked;
```

- Build a small `embarked_lookup` table: S/C/Q → full port names
- Chain as many `JOIN ... ON` clauses as you need tables
- Order matters for readability, not for correctness

7. UNION vs UNION ALL

Stacking result sets — combine rows from two queries into one result.

```
SELECT Name, 'Child' AS note FROM titanic WHERE Age < 12
UNION
SELECT Name, 'Elder' AS note FROM titanic WHERE Age > 65;
```

- `UNION` removes duplicate rows across the two result sets
- `UNION ALL` keeps duplicates and is faster since it skips the dedup step
- Both queries need the same number of columns, in the same order

8. Deliverable and push (do this when you're back home)

Write `day12_sql_intermediate.py`, reusing `titanic.db` from Day 11, running every query above and printing the results.

```
git add day12_sql_intermediate.py titanic.db
git commit -m "Day 12: SQL intermediate - subqueries, CASE, CTEs, window functions, jo
git push
```